

Revisão Ferramentas e Pipelines

Prof. Me. Deivison S. Takatu

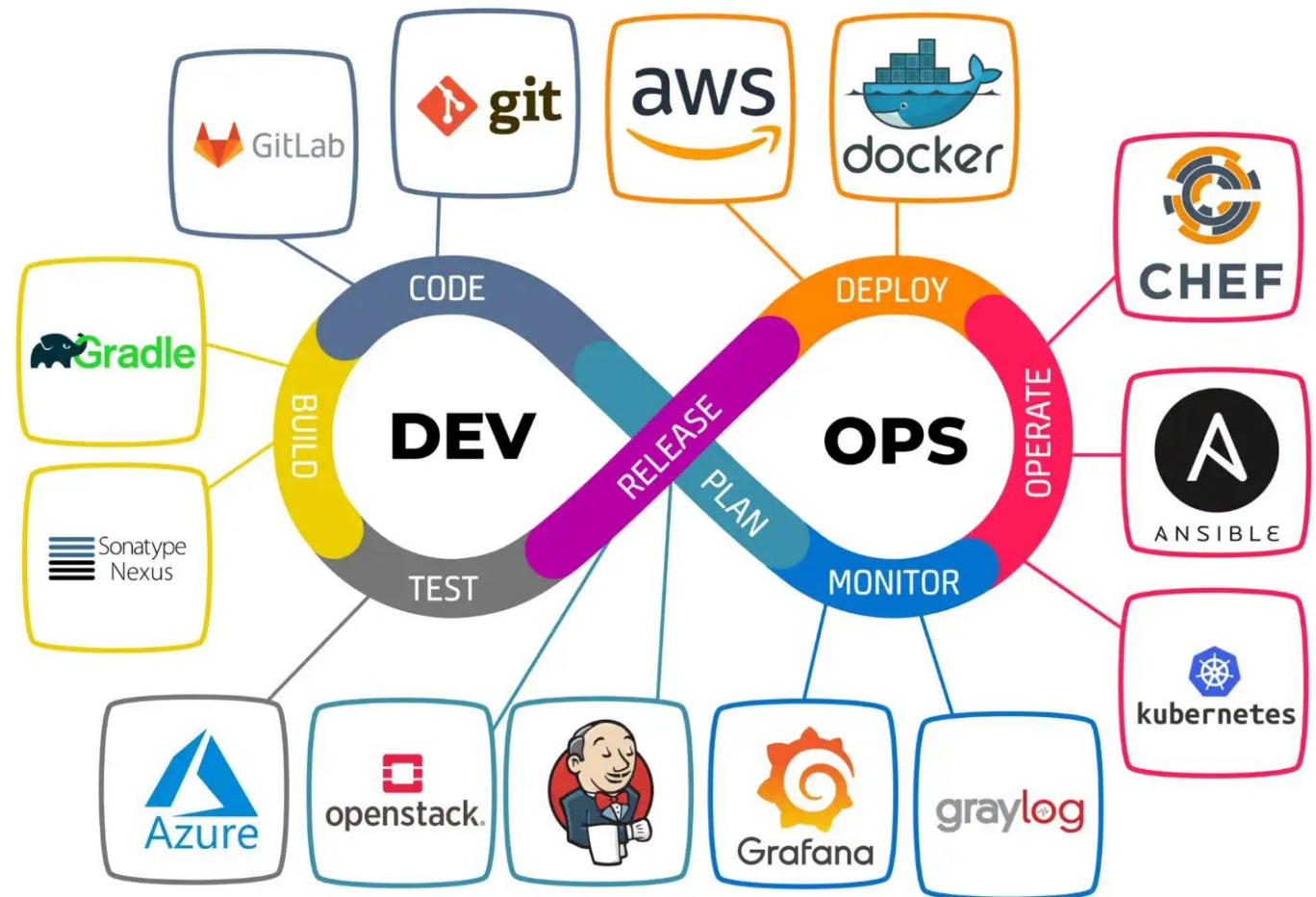
deivison.takatu@fatec.sp.gov.br

Sumário

- DevOps: O Ciclo Infinito do Desenvolvimento Moderno
- Estrutura de pipeline
- Exemplos de Pipeline
- GitHub Actions
- Entregas de Software e Automação
- Atividade.

DevOps: O Ciclo Infinito do Desenvolvimento Moderno

DevOps é uma cultura, filosofia e conjunto de práticas que integra as equipes de Desenvolvimento (DEV) e Operações (OPS), eliminando silos organizacionais e acelerando a entrega de software com qualidade e confiabilidade.



PLAN - Planejamento

O planejamento é o ponto de partida do loop infinito. Nesta etapa, as equipes definem o que será desenvolvido, como e quando, utilizando metodologias ágeis para organizar prioridades, distribuir tarefas e alinhar expectativas entre produto, desenvolvimento e operações.

- Jira Gestão de projetos ágeis com sprints, roadmaps e rastreabilidade
- Trello Quadros Kanban visuais para equipes menores e gestão de tarefas
- Azure Boards Planejamento integrado ao ecossistema Microsoft Azure DevOps

CODE - Codificação

- A etapa de codificação é onde as ideias planejadas ganham forma técnica. Nesta fase, os desenvolvedores escrevem, revisam e versionam o código-fonte de forma colaborativa, garantindo rastreabilidade, segurança e qualidade desde o início.
- Boas Práticas de Codificação no DevOps
 - Escrever código limpo e legível
 - Utilizar ferramentas de desenvolvimentos (IDEs)
 - Manter testes
 - Documentar o projeto.

BUILD - Compilação e Build Automatizado

- A etapa de Build transforma o código-fonte em um artefato executável — um arquivo JAR, imagem Docker, pacote NPM ou binário — pronto para ser testado e implantado. A automação deste processo elimina erros humanos e garante builds consistentes e reproduzíveis. Etapas:
 - Compilação: transformação do código-fonte em bytecode ou binário
 - Resolução de dependências: download automático de bibliotecas
 - Empacotamento: geração de JAR, WAR, ZIP, imagem Docker
 - Análise estática: verificação de qualidade de código (SonarQube)
 - Publicação de artefato: versionamento e armazenamento seguro

TEST - Testes Automatizados e Qualidade de Software

- Testes automatizados são o guardião da qualidade no ciclo DevOps. Ao integrar testes diretamente no pipeline de CI/CD, problemas são detectados rapidamente - antes de chegarem ao usuário final. O objetivo é garantir que cada build seja confiável, funcional e seguro.
- **Exemplo:** Uma plataforma de delivery executa +3.000 testes automatizados a cada novo deploy. O pipeline válida: cálculo de frete, processamento de pagamentos, notificações push e fluxo completo de pedido - tudo em menos de 8 minutos, sem intervenção humana.

RELEASE - Liberação e Controle de Versões

- A etapa de Release é o ponto de decisão no pipeline DevOps: o software foi desenvolvido, compilado e testado - agora precisa ser aprovado, versionado e preparado para produção. É o momento em que qualidade e governança garantem que apenas código confiável avança.
- No Release, o artefato gerado no Build recebe um número de versão semântica (ex: v2.4.1) seguindo o padrão Major.Minor.Patch. Gates de qualidade (Quality Gates) verificam cobertura de testes, análise de segurança e aprovação manual de stakeholders antes de avançar.

DEPLOY - Implantação Automatizada

- O Deploy é o momento em que o software aprovado é implantado nos ambientes de produção de forma automatizada, rápida e confiável. Com a cultura DevOps, implantações que antes levavam dias agora acontecem em minutos — com risco reduzido e reversão automática em caso de falha.
- **Exemplo:** Uma empresa de streaming possui 47 microsserviços independentes. Com Kubernetes no AWS EKS, cada serviço é implantado automaticamente via pipeline GitLab: a imagem Docker é construída, publicada no ECR (registry AWS) e o Kubernetes atualiza os pods em rolling update — sem downtime e com rollback automático se o health check falhar.;

OPERATE - Operação e Gestão de Infraestrutura

- Garante que a aplicação em produção permaneça estável, disponível e escalável. Com práticas de automação operacional, as equipes de Ops deixam de apagar incêndios manualmente e passam a gerenciar infraestrutura como código, respondendo proativamente a mudanças de demanda.
- **Exemplo:** Uma plataforma de streaming como o modelo Netflix opera com milhares de servidores globais. Durante horários de pico (novelas, jogos ao vivo), o Kubernetes aumenta automaticamente o número de réplicas dos serviços de transcodificação e entrega de conteúdo. O balanceamento de carga distribui requisições entre instâncias saudáveis, enquanto o Ansible garante configurações uniformes em todos os nós.

MONITOR - Observabilidade e Monitoramento Contínuo

O Monitoramento fecha o loop infinito do DevOps. É a etapa onde a equipe obtém visibilidade total do sistema em produção - coletando métricas, logs e traces para detectar anomalias, entender comportamentos e alimentar o próximo ciclo de planejamento com dados reais.



Métricas

Dados numéricos sobre o estado do sistema: CPU, memória, latência, taxa de erro, requisições/segundo



Logs

Registros textuais de eventos do sistema: erros, transações, autenticações e ações de usuários



Traces

Rastreamento do caminho de uma requisição através de todos os serviços envolvidos no processamento

Estrutura de pipeline

- Pipeline: Sequência de validações automatizadas executadas durante o ciclo de entrega do software.
- Build uma única vez
 - O mesmo artefato é promovido entre ambientes
- Mesmo deploy em todos os ambientes
 - Desenvolvimento → Homologação → Produção
- Falhou? Para tudo
 - Nenhuma etapa continua após erro

Um desenvolvedor altera a tela de login de um sistema e envia o código para o GitHub. A partir disso, o pipeline inicia automaticamente:

1. O sistema baixa o código atualizado
2. Compila a aplicação
3. Executa testes automáticos
4. Verifica a qualidade do código
5. Gera uma nova versão da aplicação
6. Pública em ambiente de homologação

Pipeline de build

- O pipeline de build é a etapa responsável por preparar automaticamente o software para execução ou distribuição.
- O objetivo é garantir que o sistema consiga ser construído corretamente antes das próximas etapas do pipeline.
- Principais funções
 - Compilação automatizada do projeto
 - Geração de artefatos de software
 - arquivos .jar, .war, .apk, containers Docker etc.
 - Verificação inicial da integridade do código

Pipeline de testes

- O pipeline de testes executa automaticamente validações no sistema após o processo de build.
- O objetivo é garantir que novas alterações não quebrem funcionalidades existentes do sistema.
- Principais funções
 - Execução automática de testes
 - Validação contínua da aplicação
 - Identificação rápida de falhas

Pipeline de qualidade

- O pipeline de qualidade realiza verificações automatizadas para avaliar se o código atende aos padrões definidos antes de avançar para as próximas etapas.
- O objetivo é identificar problemas de qualidade, inconsistências e possíveis falhas no código de forma antecipada.

Pipeline de segurança

- O pipeline de segurança realiza verificações automatizadas para identificar vulnerabilidades e riscos no software durante o processo de desenvolvimento.
- Principais funções
 - Análise de vulnerabilidades no código
 - Verificação de dependências utilizadas
 - Identificação de configurações inseguras
 - Detecção de informações sensíveis expostas

Pipeline de Artefatos (Package)

- O pipeline de artefatos é responsável por empacotar e armazenar os resultados gerados durante o processo de build.
- Principais funções
 - Empacotamento da aplicação
 - Geração de arquivos de distribuição
 - Versionamento dos artefatos
 - Armazenamento em repositórios de artefatos

Pipeline de Deploy

- O pipeline de deploy é responsável por publicar automaticamente o software em um ambiente de execução após as etapas de validação.
- Principais funções
 - Preparação do ambiente de destino
 - Publicação do artefato gerado
 - Configuração de variáveis e parâmetros
 - Execução do deploy automatizado

Pipeline de Release e Monitoramento

- O pipeline de release é responsável por preparar e disponibilizar uma versão específica do software para os usuários ou para um ambiente de produção.
- Principais funções
 - Criação e identificação de versões
 - Publicação de releases
 - Controle das versões disponibilizadas
 - Monitoramento da aplicação em produção

Exemplo de Projeto

- Projeto prático utilizando GitHub Actions para automatizar uma pipeline de CI/CD.
- Cada etapa é executada automaticamente e uma falha interrompe o fluxo.
- Link: github.com/deivisontakatu/projeto-pipelines-devops

GitHub Actions

É uma plataforma de automação integrada ao GitHub, permitindo criar e executar fluxos de trabalho diretamente associados aos repositórios de código.

Os workflows podem ser configurados de acordo com eventos do repositório, como push, pull request, criação de tags ou releases.

GitHub Actions Marketplace

- É o catálogo de Actions, ferramentas e extensões que podem ser utilizadas nos workflows do GitHub Actions.
- Permite encontrar soluções prontas para build, testes, qualidade, segurança, deploy, cache e outras etapas da pipeline.
- As Actions podem ser desenvolvidas pelo GitHub, parceiros ou pela comunidade.
- GitHub Marketplace: github.com/marketplace?type=actions

Entregas de Software e Automação

DevOps

Integra desenvolvimento, testes e operações por meio da automação, acelerando entregas de software com maior qualidade, estabilidade e redução de falhas.

Definição de Pipeline

Sequência automatizada de estágios responsável por conduzir o software desde o commit até o deploy de forma confiável.

Humble e Priklandnicki (2013).

O Papel das Pipelines

As pipelines de CI tornaram-se fundamentais no desenvolvimento moderno, automatizando validações e garantindo qualidade contínua durante todo o ciclo de entrega.



“Pipelines promovem qualidade contínua ao longo de todo o processo de entrega” SATO (2014).

Entregas de Software e Automação

Gatilhos (Workflows)

Iniciam automaticamente em ações como:

- Push de código
- Pull Requests
- Criação de Releases
- Publicação de Tags

É uma ferramenta nativa do GitHub que permite automatizar processos de integração contínua, testes e deploy de aplicações.

```
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
```

Exemplo: Execução da pipeline DevOps com GitHub Actions.

Atividade

Integrem uma pipeline ao projeto utilizando o GitHub Actions, configurando a automação para que seja executada sempre que for realizado um novo push na branch principal (main).

Em seguida, busquem repositórios no Github que contenham a integração de Pipeline e realizem uma análise de pelo menos três projetos, destacando características, funcionalidades, gatilhos e histórico.

Referências

- HUMBLE J; PRIKLANDNICKI R. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013
- MUNIZ, A.; et al. Jornada Devops: Unindo Cultura Ágil, Lean e Tecnologia Para Entrega De Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. DevOps na prática: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. Entrega contínua em Android: Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.
- ARUNDEL, J. DOMINGUS, J. DevOps nativo de nuvem com Kubernetes. São Paulo: Novatec, 2019.

Referências

- MORAES, G. Caixa de Ferramentas DevOps: Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015.
- PIRES, A.; MILITÃO, J. Integração Contínua com Jenkins. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. Descomplicando o Docker. 2 ed. São Paulo: Brasport, 2018.
- SILVERMAN, R. E. Git: guia prático. São Paulo: Novatec, 2019.
- KIM, G; HUMBLE, J; DEBOIS, P; WILLIS, J; Manual de DEVOPS: Como obter agilidade, confiabilidade e segurança em organizações tecnológicas. São Paulo: Starlin Alta Editora, 2018.

Revisão Ferramentas e Pipelines

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br